Inheritance and Polymorphism in Python

What is Inheritance?

- **Inheritance** allows a class (child/derived class) to reuse properties and methods of another class (parent/base class).
- It helps in **code reusability** and **method extension**.

Types of Inheritance in Python:

1. **Single Inheritance** → One child class inherits from one parent class.
2. **Multilevel Inheritance** → A class inherits from another derived class.
3. **Hierarchical Inheritance** → Multiple child classes inherit from the same parent.
4. **Multiple Inheritance** → A child class inherits from more than one parent class.

Example 1: Basic Inheritance

```
class A:
    def m1(self):
        print("This is m1 method from class A")


class B(A):  # B inherits from A
    def m2(self):
        print("This is m2 method from class B")


bobj = B()
bobj.m1()  # Parent method
bobj.m2()  # Child method
```

- Child class B can access both its own method and parent's method.

Example 2: Single Inheritance

```
class A:
    x,y = 10,20
    def m1(self):
        print(self.x + self.y)

class B(A):
    a,b = 200,100
    def m2(self):
        print(self.a - self.b)

bobj = B()
bobj.m1()  # 30
bobj.m2()  # 100
```

- Child class can access parent's variables and methods.

Example 3: Multilevel Inheritance

```
class A:
    x,y = 10,20
    def m1(self):
        print(self.x + self.y)

class B(A):
    a,b = 200,100
    def m2(self):
        print(self.a - self.b)

class C(B):
    i,j = 5,2
```

```
    def m3(self):
        print(self.i * self.j)

cobj = C()
cobj.m1()  # 30
cobj.m2()  # 100
cobj.m3()  # 10
```

- Chain of inheritance: A → B → C.

Example 4: Hierarchical Inheritance

```
class A:
    x,y = 10,20
    def m1(self):
        print(self.x + self.y)

class B(A):
    a,b = 200,100
    def m2(self):
        print(self.a - self.b)

class C(A):
    i,j = 5,2
    def m3(self):
        print(self.i * self.j)

bobj = B()
bobj.m1()  # 30
bobj.m2()  # 100
```

```
cobj = C()
cobj.m1()  # 30
cobj.m3()  # 10
```

- Both B and C share the same parent A.

Example 5: Multiple Inheritance

```
class A:
    x,y = 10,20
    def m1(self):
        print(self.x + self.y)

class B:
    a,b = 200,100
    def m2(self):
        print(self.a - self.b)

class C(A,B):   # Inherits from both A & B
    i,j = 5,2
    def m3(self):
        print(self.i * self.j)

cobj = C()
cobj.m1()  # 30
cobj.m2()  # 100
cobj.m3()  # 10
```

- C can access features of both A and B.

Example 6: Using super()

```
class A:
    def m1(self):
        print("This is m1 method from class A.....")

class B(A):
    def m1(self):
        print("This is m1 method from class B.....")
        super().m1()  # calling parent method

bobj = B()
bobj.m1()
```

- super() is used to call the **parent class method** from the child class.

Example 7: Accessing Parent Variables

```
class A:
    a,b = 10,20

class B(A):
    i,j = 100,200
    def m(self, x, y):
        print(x+y)          # Local variables
        print(self.i+self.j) # Child class variables
        print(self.a+self.b) # Parent class variables

bobj = B()
bobj.m(1000,2000)
```

- A child class can access local variables, its own variables, and parent's variables.

What is Polymorphism?

- **Polymorphism** means "many forms".
- In Python, it allows the **same method name** to behave differently based on context.

Types of Polymorphism:

1. **Method Overriding** → Child class provides its own version of a method from the parent.
2. **Method Overloading** → Same method name but with different numbers of parameters (Python does not support true overloading but can simulate it using default arguments).

Example 8: Variable Overriding

```
class Parent:
    name = "scott"


class Child(Parent):
    name = "John"  # overriding variable


childobj = Child()
print(childobj.name)  # John
```

- Child class variable overrides parent class variable.

Example 9: Method Overriding

```
class Bank:
    def rateOfInterest(self):
        return 0


class XBank(Bank):
    def rateOfInterest(self):
        return 10.5
```

```
class YBank(Bank):
    def rateOfInterest(self):
        return 12.5

x = XBank()
print(x.rateOfInterest())  # 10.5

y = YBank()
print(y.rateOfInterest())  # 12.5
```

- Same method rateOfInterest() behaves differently in each child class.

Example 10: Method Overloading (Simulated)

```
class Human:
    def sayHello(self, name=None):
        if name is not None:
            print("Hello " + name)
        else:
            print("Hello")

h = Human()
h.sayHello()        # Hello
h.sayHello("David")  # Hello David
```

- Python achieves method overloading by using **default arguments**.

Example 11: Another Overloading Example

```
class Calculation:
    def add(self, a=0, b=0, c=0):
        print(a + b + c)
```

```
calobj = Calculation()
calobj.add()            # 0
calobj.add(10, 20)      # 30
calobj.add(100,200,300)  # 600
```

- Same method works with different number of arguments.

Summary

- **Inheritance** → Reuse parent class methods/variables.
  - Types: Single, Multilevel, Hierarchical, Multiple.
  - Use super() to access parent methods/variables.
- **Polymorphism** → Same name, different behaviors.
  - Variable overriding.
  - Method overriding (child redefines parent method).
  - Method overloading (using default arguments).